skill评估优化

1. skill创建

Skill Creator 将技能创建分为五个关键阶段，形成一个循环：

1. **确定意图 (Decide)**：明确技能要做什么，触发场景是什么。
2. **编写草案 (Draft)**：编写 `SKILL.md` 文件，包含 YAML 元数据和 Markdown 指令。
3. **运行测试 (Test)**：创建测试用例，同时运行“有技能辅助”和“无技能（基准）”的对比实验。
4. **评估结果 (Evaluate)**：
   - **定性**：通过 `eval-viewer` 生成可视化界面，让用户亲自看结果并写反馈。
   - **定量**：运行断言（Assertions），计算通过率、耗时和 Token 消耗。
5. **迭代优化 (Iterate)**：根据反馈修改技能，再次运行测试，直到满意。

代码块

```
recsys-workflow-skill/
├── SKILL.md                          # 主编排逻辑（324行）
└── references/
    ├── phase2_math_templates.md      # 数学建模模板库
    └── phase3_code_templates.md      # 工程代码模板库
```

写测试用例（Evals）- 跑基准测试（Benchmark）- 对比不同版本（A/B Test）- 自动优化描述（Trigger Tuning）

大致评估优化内容： 主要两类：结构类和内容类

1. 功能正确性：能不能**完成预设任务**
2. 意图识别准确率：用户 query 能不能**正确触发该 Skill**
3. 响应质量：答案准确、完整、简洁
4. 交互与上下文能力：多轮对话是否记得上文
5. 性能指标：

调用成功率

平均响应时延

资源占用、稳定性

6. 安全与合规

方法：自动 和 llm-judge：

1. 自动化评估：写**测试用例集**（正常 / 异常 / 边界），批量跑 Skill
7. LLM-as-Judge（大模型当裁判，现在最主流）
8. 人工评估
9. 线上 A/B 测试
10. 边界 & 对抗鲁棒性测试
11. 基准数据集 / 标准化评测

2. 优化方法 (Optimization Methods)

Skill 的优化分为内部逻辑优化和外部触发机制优化。

A. 内容与逻辑优化

1.从反馈中总结经验。从反馈中泛化（避免过拟合）

2.保持提示简洁。精简 Prompt

3.解释原因。解释“为什么”要这么做以利用大模型的心智理论

4.寻找测试用例中的重复操作。将重复的工作封装为 `scripts/` 中的独立脚本。

人类或 Analyzer 的诊断反馈数据:

具体实现：

对于客观的，打断言，测试结构正确性；主观的，llm jude

没有输出doc文件，使用skill-creator优化这个上面那个pdf总结的skill

#	断言	旧版	新版
1	含 □ 特殊字符的 PDF 能成功生成 docx	❌ 崩溃	✅ 40KB
2	含数学符号 \ud835 的 PDF 能成功生成 docx	❌ 崩溃	✅ 38KB
3	生成的 docx 含 Word 标题样式（Heading 1/2/3）	❌ 根本没文件	✅ H1/H2/H3
4	生成的 docx 含列表样式（List Bullet）	❌ 根本没文件	✅ 有

新版通过率：4/4（100%） | 旧版通过率：0/4（0%）

图片是目前新版输出的摘要里面的内容，对表格样式的支持不是很好，同时摘要里面将创新点放在了最后，我觉得这样的排版不是很合适。请优化。使用skill-creator进行优化前后对比

断言	新版	旧版
docx 成功生成	✅	✅
含真实 Word 表格	✅ 1个，7行×2列	❌ 0个
表格无原始竖线字符	✅	✅（但根本没写表格，不算真正通过）
创新点位置 ≤ 第4节	✅ 第2节	❌ 第8节

代码块

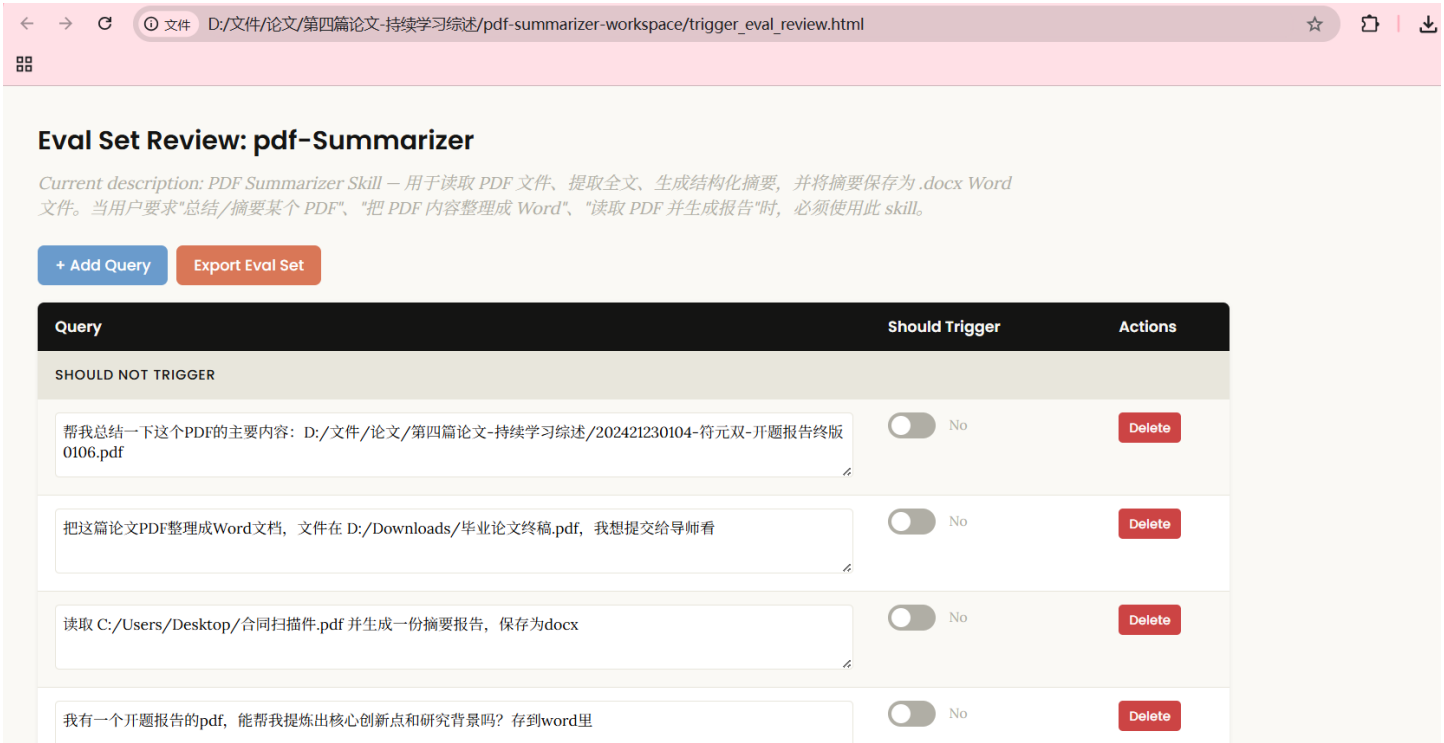
```
{
  "reviews": [
    {
      "run_id": "eval-2-with_skill",
      "feedback": "关于方案A和方案B的架构对比，现在的文档里只有文字说明，大模型在回答时经常忘记强调数据泄露防护（Target-aware attention），需要把 Attention Mask 的矩阵规则明确写出来。",
      "timestamp": "2026-03-23T10:15:30Z"
    },
    {
      "run_id": "eval-5-with_skill",
      "feedback": "面对'禁止Ensemble下如何提升'的问题，现在的 Skill 完全不知所措，因为它只有约束规则，没有提供'怎么做'的替代方案列表。",
      "timestamp": "2026-03-23T10:18:45Z"
    }
  ],
  "status": "complete"
}
```

B. 触发描述自动优化 (Description Optimization)

`在优化 Description 时，测试 Claude 遇到各种模糊或相似的提问时，能否准确地“唤醒”并调用 Skill。`

- **测试集构建**：生成约 20 个查询，混合“应触发 (should-trigger)”和“不应触发 (should-not-trigger，通常是具有迷惑性的相邻领域查询)”的用例。通过 `assets/eval_review.html` 让用户确认测试集。

```
工作原理：
①准备 20 条查询（10 条应触发 + 10 条不应触发）
②对每条查询运行 claude -p，检测是否调用了 Skill 工具
③计算 precision/recall/accuracy
④把失败案例传给 Claude，要求生成改进后的 description
⑤重新评估新描述 → 循环最多 5 次
⑥以测试集（holdout 40%）得分选出最优描述
```



整个流程：

第一步：把当前的 Skill 描述（比如“这是一个推荐系统工具”）丢进 `run_eval` 去跑训练集和测试集。

第二步： 如果训练集全过（`train_summary["failed"] == 0`），直接宣布胜利（`all_passed`），跳出循环。

第三步：错题本与反思。 如果有没通过的 case，——它调用了外部脚本 `improve_description`。它会把当前的错误结果（哪些该触发没触发，哪些不该触发误触发了）打包，送给另一个大模型（通常是 Claude 3.5 Sonnet 或 Opus）。

第四步：重写规则。 大模型看完“错题本”后，会总结失败原因，并生成一段**全新的 Skill Description**。然后带着这个新描述，进入下一轮迭代考试。

大模型的输出是概率分布的（非确定性状态机）。哪怕是完全相同的 Prompt，Claude 可能第一次决定调用 Skill，第二次却决定自己直接回答。

统计学评估：脚本为每一个查询分配了并行任务（`ProcessPoolExecutor`）。默认情况下，同一个 Prompt 会跑 3 次。如果触发次数 / 总运行次数 >=0.5（即至少触发了 2 次），才算该测试用例

`PASS`。

正例与反例测试：注意代码里的 `should_trigger` 变量。这个评估框架不仅测试“该触发时有没有触发”，还会测试“不该触发时是不是误触发了”。这对于防止工具过度活跃（Over-triggering）至关重要

代码块

```
[
  {
    "query": "我刚看到腾讯广告的推荐系统赛题，赛题是'Towards Unifying Sequence Modeling'，能帮我解释一下这道题到底要解决什么问题吗？",
    "should_trigger": true
  },
  {
    "query": "推荐系统竞赛中，如何把序列建模和特征交互融合在一个Recommendation Block里？",
    "should_trigger": true
  },
  {
    "query": "帮我设计一个实验，验证推荐系统中单一Transformer Block的Scaling Law曲线，最大参数量要测到large级别。",
    "should_trigger": true
  },
  {
    "query": "我想写一个推荐系统，用协同过滤算法（Collaborative Filtering）怎么给用户推荐商品？",
    "should_trigger": false,
    "_comment": "负样本：虽然是推荐系统，但是通用基础算法，不涉及本赛题的序列与特征统一，不应触发该高级Skill。"
  },
  {
    "query": "帮我总结一下《Attention is All You Need》这篇论文里Transformer的位置编码是怎么设计的。",
    "should_trigger": false,
    "_comment": "负样本：虽然提到了Transformer和位置编码，但属于通用深度学习知识，与推荐赛题无关。"
  },
  {
    "query": "腾讯广告的点击率预估业务中，一般怎么处理缺失值？",
    "should_trigger": false,
    "_comment": "负样本（高难度干扰项）：提到了腾讯广告，但属于通用CTR预估实战，除非用户明确提及赛题或Recommendation Block，否则不应触发。"
  }
```

```
]
```

3. 评估：

1.

评估环节分为**定量自动评估**、**定性人工评估**以及**盲测对比**，这是整个框架最核心的质量把控部分。

A. 自动定量评分 (Automated Grading)

- **执行与评估文件**：由 Grader Agent 根据 `agents/grader.md` 提取测试输出并进行评分。其结果会输出为 `grading.json`。
- **评估指标**：
  - **通过率 (Pass Rate)**：基于客观断言的通过比例（`passed` / `total`）：

    量化方式：大模型阅卷（单题判定）+（计算比例）。大模型如何评分（定性判决）： 系统会生成一个叫 Grader 的大模型 Agent，它会拿着写好的断言（Expectations）去对比生成的输出和运行日志。在 `skill-creator/agents/grader.md` 文件中，写死了大模型评分的 Prompt 指令：

断言	iter-3	iter-4
docx 文件成功生成（开题报告）	✅ PASS	✅ PASS
文档含真实 Word 表格（开题报告）	✅ PASS（1个）	✅ PASS（2个）
无原始竖线字符段落（开题报告）	✅ PASS	✅ PASS
核心创新点 ≤ 第4节（开题报告）	✅ PASS（第2节）	✅ PASS（第2节）
docx 文件成功生成（引言）	✅ PASS	✅ PASS
文档含 Word 标题样式（引言）	✅ PASS	✅ PASS
文档含列表样式（引言）	✅ PASS	✅ PASS
提取不崩溃（引言）	✅ PASS	✅ PASS
总通过率	8/8（100%）	8/8（100%）

三次优化的通过率演进

```
v1（原始）        v2（编码修复）      v3（表格+顺序）      v3 再次运行
 0/6   →          6/6      →          4/4       →       8/8
 (0%)             (100%)              (100%)             (100%)
```

`PASS when: The transcript or outputs clearly demonstrate the expectation is true...`（当日志或输出明确证明预期为真时给予 PASS）

`FAIL when: No evidence found for the expectation...`（找不到证据或只是表面满足时给予 FAIL） Grader Agent 评判后，会将每条断言的 True/False 结果保存在 `grading.json` 中。

代码块

```
1  {
2    "expectations": [
3      {
4        "text": "The output includes the name 'John Smith'",
5        "passed": true,
6        "evidence": "Found in transcript Step 3: 'Extracted names: John
   Smith, Sarah Johnson'"
7      },
8      {
9        "text": "The spreadsheet has a SUM formula in cell B10",
10       "passed": false,
11       "evidence": "No spreadsheet was created. The output was a text file."
12     },
13     {
```

```
      "text": "The assistant used the skill's OCR script",
      "passed": true,
      "evidence": "Transcript Step 2 shows: 'Tool: Bash - python ocr_script.py image.png'"
    }
  ],
  "summary": {
    "passed": 2,
    "failed": 1,
    "total": 3,
    "pass_rate": 0.67
  },
  "execution_metrics": {
    "tool_calls": {
      "Read": 5,
      "Write": 2,
      "Bash": 8
    },
    "total_tool_calls": 15,
    "total_steps": 6,
    "errors_encountered": 0,
    "output_chars": 12450,
    "transcript_chars": 3200
  },
  "timing": {
    "executor_duration_seconds": 165.0,
    "grader_duration_seconds": 26.0,
    "total_duration_seconds": 191.0
  },
  "claims": [
    {
      "claim": "The form has 12 fillable fields",
      "type": "factual",
      "verified": true,
      "evidence": "Counted 12 fields in field_info.json"
    },
    {
      "claim": "All required fields were populated",
      "type": "quality",
      "verified": false,
      "evidence": "Reference section was left blank despite data being available"
    }
  ],
  "user_notes_summary": {
    "uncertainties": ["Used 2023 data, may be stale"],
    "needs_review": [],
```

```
      "workarounds": ["Fell back to text overlay for non-fillable fields"]
    },
    "eval_feedback": {
      "suggestions": [
        {
          "assertion": "The output includes the name 'John Smith'",
          "reason": "A hallucinated document that mentions the name would also pass — consider checking it appears as the primary contact with matching phone and email from the input"
        },
        {
          "reason": "No assertion checks whether the extracted phone numbers match the input — I observed incorrect numbers in the output that went uncaught"
        }
      ],
      "overall": "Assertions check presence but not correctness. Consider adding content verification."
    }
  }
```

代码如何量化（定量计算）： 在 `skill-creator/scripts/aggregate_benchmark.py` 脚本中，Python 代码会读取这些 True/False，计算单次运行的通过率，并使用 `calculate_stats()` 函数计算所有运行轮次的平均值和标准差

- **时间 (Time)**：系统会捕获任务耗时，存储在 `timing.json` 中的 `total_duration_seconds`。

迭代	测试用例	配置	耗时
iteration-1	eval-encoding-special-chars	with_skill	106.5 秒
iteration-1	eval-format-quality	with_skill	74.0 秒
iteration-1	old_skill（两个）	–	崩溃（几秒内报错）
iteration-4	eval-kaiti-report	with_skill	91.0 秒
iteration-4	eval-yinyan	with_skill	54.0 秒

- **Token消耗 (Tokens)**：总 Token 使用量或输出字符数 (`output_chars`)。

迭代	测试用例	配置	Token
iteration-1	eval-encoding-special-chars	with_skill（新版）	40,000
iteration-1	eval-format-quality	with_skill（新版）	25,938
iteration-1	eval-encoding-special-chars	old_skill（旧版）	崩溃，未完成
iteration-1	eval-format-quality	old_skill（旧版）	崩溃，未完成
iteration-2	-	-	未记录（手动运行）
iteration-3	eval-kaiti-report	with_skill	0（未记录）
iteration-3	eval-yinyan	with_skill	0（未记录）
iteration-4	eval-kaiti-report	with_skill	37,869
iteration-4	eval-yinyan	with_skill	15,905

当每个 subagent 任务完成时，你会收到一个包含 `total_tokens` 和 `duration_ms` 的通知。必须立即将此数据保存到运行目录中的 `timing.json` 中”。这是一个只出现一次的底层系统事件回调。

- **工具调用与错误**：总工具调用次数（`tool_calls`）以及执行中遇到的错误数（`errors_encountered`）。

Agent 框架（Executor）在运行过程中真实记录的统计数据。

当大模型执行任务时，它每一次调用工具（如 Read、Write、Bash 等）和发生的报错，都会被执行环境捕获，并写入到运行目录下的 `outputs/metrics.json` 文件中

- **自动提取声明 (Claims Verification)**：Grader 还会自动从输出中提取事实声明、过程声明和质量声明，并尝试验证其真伪。

代码块

```
cat > /home/claude/benchmark-workspace/eval-1/with_skill/run-1/outputs/response.md << 'RESP'
# 赛题解析：推荐系统为什么"分道扬镳"

根据赛题背景，核心问题在于工业推荐系统过去二十年形成了两条**完全独立**的技术路线：

**序列建模路线**（代表：DIN、DIEN、SIM、GRU4Rec、BERT4Rec）
- 专注于用户历史行为的时序依赖
- 建模"用户昨天点了A，今天更可能点B"的序列模式
- 缺陷：只关注序列信号，难以和用户画像、物料属性等静态特征深度结合

**特征交互路线**（代表：FM、DeepFM、DCN、AutoInt、xDeepFM）
- 专注于多域离散特征的组合（如"年龄×品类"的协同效应）
- 建模"25岁女性更喜欢美妆"这类特征交叉规律
- 缺陷：只看当前时刻的特征快照，完全忽视时序动态

```

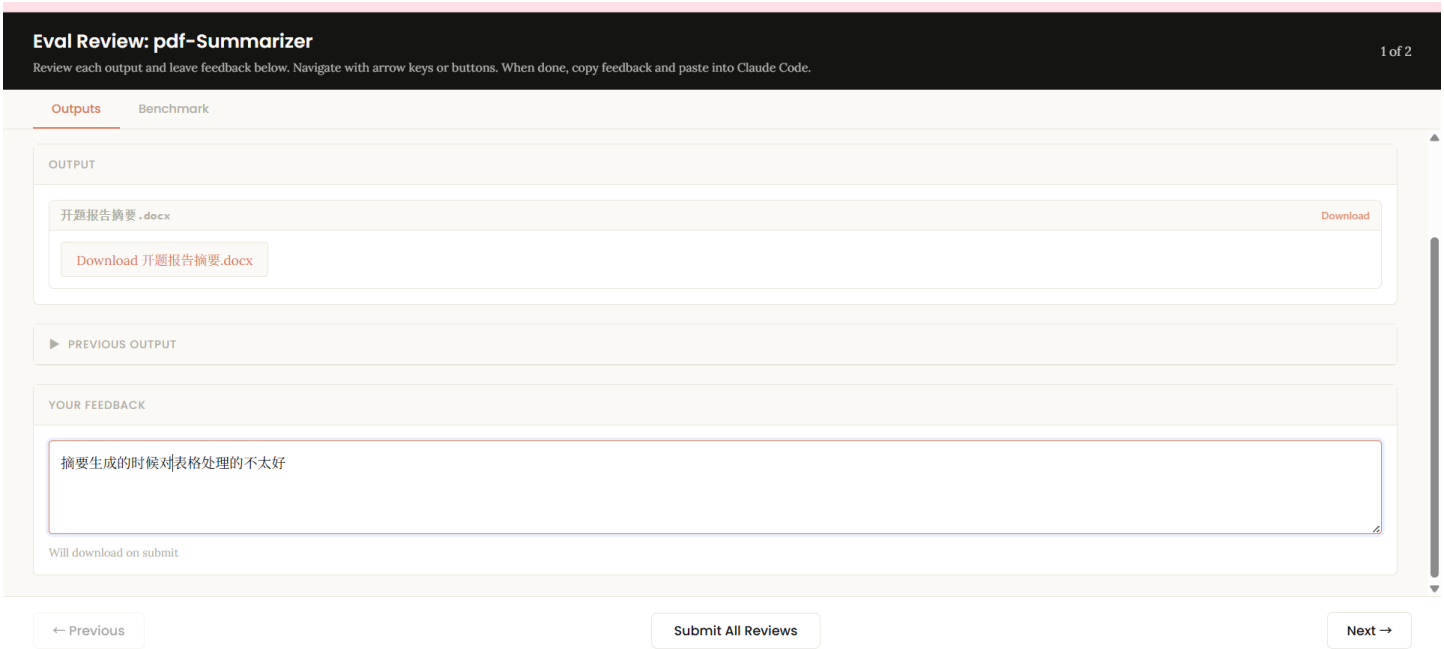
```
**"分道扬镳"的工程后果**:
两条路线形成**异构子网络**，带来三个严重问题:
1. **算力碎片化**: 两套参数需要独立 GPU/内存资源，利用率低
2. **交互浅薄**: 序列 embedding 和特征 embedding 通常只在最后一层 concat，相当于"相亲但不结婚"
3. **无法统一 Scaling**: 无法像 LLM 一样探索统一参数规模的幂律增益

**赛题要求**: 设计**单一同构的 Recommendation Block**，让这两路信号在同一架构内实现**深层**交互，同时探索推荐模型的 Scaling Law。

约束: 全程禁止 Ensemble，只能用单一架构的实力说话。
RESP

cat > /home/claude/benchmark-workspace/eval-1/with_skill/run-1/outputs/metrics.json << 'EOF'
{
  "tool_calls": {"Read": 1, "Write": 1},
  "total_tool_calls": 2,
  "total_steps": 2,
  "files_created": ["response.md"],
  "errors_encountered": 0,
  "output_chars": 978,
  "transcript_chars": 420
}
EOF

cat > /home/claude/benchmark-workspace/eval-1/with_skill/run-1/timing.json << 'EOF'
{
  "total_tokens": 2140,
  "duration_ms": 8200,
  "total_duration_seconds": 8.2,
  "executor_duration_seconds": 7.8,
  "grader_duration_seconds": 12.1
}
EOF
echo "eval-1 with_skill outputs written"
```

该脚本会读取所有的 `grading.json` 和 `timing.json`，计算每个配置（有 Skill vs 无 Skill）下各项指标的**均值 (mean)**、**标准差 (stddev)**、**最小值**和**最大值**，并计算两者之间的 **Delta (差值)**。最终生成 `benchmark.json` 和可视化的 `benchmark.md`。

C. 定性人工评估 (Qualitative Human Review)

- **执行文件**: `eval-viewer/generate_review.py` 和对应的 `viewer.html`。

- **方法**：脚本会启动一个本地的 HTML 浏览器视图，将提示词、生成的输出文件、格式化成绩和基准测试统计并排展示。用户可以通过 UI 点击浏览各个输出，并在文本框中留下主观反馈，反馈会实时自动保存到工作区的 `feedback.json` 中。



D. 盲测对比与深度分析 (Blind Comparison - 高级评估)

- **比较方法**：由 Blind Comparator Agent (`agents/comparator.md`) 在不知道哪个输出是由哪个配置生成的情况下，对两个输出（A和B）进行打分。
- **评分量表 (Rubric, 1-5分)**：
  - **内容 (Content)**：正确性 (Correctness)、完整性 (Completeness)、准确性 (Accuracy)。
  - **结构 (Structure)**：组织 (Organization)、格式 (Formatting)、可用性 (Usability)。
- **结果分析**：Post-hoc Analyzer Agent (`agents/analyzer.md`) 会在比较结束后“揭盲”，分析胜者获胜的原因，对“指令遵循度 (Instruction Following)”进行 1-10 分的打分，并针对失败者提出具体的改进建议（涉及指令、工具、示例、错误处理等分类）。

Content Rubric (what the output contains):

Criterion	1 (Poor)	3 (Acceptable)	5 (Excellent)
Correctness	Major errors	Minor errors	Fully correct
Completeness	Missing key elements	Mostly complete	All elements present
Accuracy	Significant inaccuracies	Minor inaccuracies	Accurate throughout

Structure Rubric (how the output is organized):

Criterion	1 (Poor)	3 (Acceptable)	5 (Excellent)
Organization	Disorganized	Reasonably organized	Clear, logical structure
Formatting	Inconsistent/broken	Mostly consistent	Professional, polished
Usability	Difficult to use	Usable with effort	Easy to use

维度	A（iter-3）	B（iter-4）	胜者
C1 核心创新点位置	5	5	TIE
C2 研究计划表格	3	3	TIE
C3 信息覆盖完整性	4	5	B
C4 章节结构	3	3	TIE
C5 内容丰富度	4	5	B
总分	19	21	B 胜

B 胜的原因：多了 θ=0.85 参数、匈牙利算法/CVAE/PPO
名称、技术栈（FastAPI/Vue.3）、研究现状综述段落。两个版本的共同缺陷：研究

引言 盲测（A=iter-3, B=iter-4）

维度（权重）	A（iter-3）	B（iter-4）	胜者
C1 创新点覆盖（25%）	5	4	A
C2 章节结构（20%）	4	5	B
C3 内容完整性（25%）	5	3	A
C4 列表结构化（15%）	5	4	A
C5 研究计划（15%）	3	4	B
加权得分	4.50	3.95	A 胜

代码块

```
{
  "winner": "A",
  "reasoning": "Output A provides a complete solution with proper formatting and all required fields. Output B is missing the date field and has formatting inconsistencies.",
  "rubric": {
    "A": {
      "content": {
        "correctness": 5,
        "completeness": 5,
        "accuracy": 4
      },
      "structure": {
        "organization": 4,
        "formatting": 5,
        "usability": 4
      },
      "content_score": 4.7,
      "structure_score": 4.3,
      "overall_score": 9.0
    },
    "B": {
      "content": {
        "correctness": 3,
        "completeness": 2,
        "accuracy": 3
      },
      "structure": {
        "organization": 3,
        "formatting": 2,
        "usability": 3
      },
      "content_score": 2.7,
      "structure_score": 2.7,
      "overall_score": 5.4
    }
  },
  "output_quality": {
    "A": {
      "score": 9,
      "strengths": ["Complete solution", "Well-formatted", "All fields present"],
      "weaknesses": ["Minor style inconsistency in header"]
    },
    "B": {
      "score": 5,
      "strengths": ["Readable output", "Correct basic structure"],
```

```
45          "weaknesses": ["Missing date field", "Formatting inconsistencies",
    "Partial data extraction"]
46        }
47      },
48      "expectation_results": {
49        "A": {
50          "passed": 4,
51          "total": 5,
52          "pass_rate": 0.80,
53          "details": [
54            {"text": "Output includes name", "passed": true},
55            {"text": "Output includes date", "passed": true},
56            {"text": "Format is PDF", "passed": true},
57            {"text": "Contains signature", "passed": false},
58            {"text": "Readable text", "passed": true}
59          ]
60        },
61        "B": {
62          "passed": 3,
63          "total": 5,
64          "pass_rate": 0.60,
65          "details": [
66            {"text": "Output includes name", "passed": true},
67            {"text": "Output includes date", "passed": false},
68            {"text": "Format is PDF", "passed": true},
69            {"text": "Contains signature", "passed": false},
70            {"text": "Readable text", "passed": true}
71          ]
72        }
73      }
74    }
```

代码块

```
1   {
2     "comparison_summary": {
3       "winner": "A",
4       "winner_skill": "path/to/winner/skill",
5       "loser_skill": "path/to/loser/skill",
6       "comparator_reasoning": "Brief summary of why comparator chose winner"
7     },
8     "winner_strengths": [
9       "Clear step-by-step instructions for handling multi-page documents",
10      "Included validation script that caught formatting errors",
```

```
      "Explicit guidance on fallback behavior when OCR fails"
    ],
    "loser_weaknesses": [
      "Vague instruction 'process the document appropriately' led to inconsistent behavior",
      "No script for validation, agent had to improvise and made errors",
      "No guidance on OCR failure, agent gave up instead of trying alternatives"
    ],
    "instruction_following": {
      "winner": {
        "score": 9,
        "issues": [
          "Minor: skipped optional logging step"
        ]
      },
      "loser": {
        "score": 6,
        "issues": [
          "Did not use the skill's formatting template",
          "Invented own approach instead of following step 3",
          "Missed the 'always validate output' instruction"
        ]
      }
    },
    "improvement_suggestions": [
      {
        "priority": "high",
        "category": "instructions",
        "suggestion": "Replace 'process the document appropriately' with explicit steps: 1) Extract text, 2) Identify sections, 3) Format per template",
        "expected_impact": "Would eliminate ambiguity that caused inconsistent behavior"
      },
      {
        "priority": "high",
        "category": "tools",
        "suggestion": "Add validate_output.py script similar to winner skill's validation approach",
        "expected_impact": "Would catch formatting errors before final output"
      },
      {
        "priority": "medium",
        "category": "error_handling",
        "suggestion": "Add fallback instructions: 'If OCR fails, try: 1) different resolution, 2) image preprocessing, 3) manual extraction'",
        "expected_impact": "Would prevent early failure on difficult documents"
      }
```

```
  ],
  "transcript_insights": {
    "winner_execution_pattern": "Read skill -> Followed 5-step process -> Used validation script -> Fixed 2 issues -> Produced output",
    "loser_execution_pattern": "Read skill -> Unclear on approach -> Tried 3 different methods -> No validation -> Output had errors"
  }
}
```

论文：

- **通过率 (Pass Rate)**：这是最核心的基础指标。为了减小随机性，基准测试对每个任务会进行 5 次独立跑分取平均，最后再将所有 84 个评估任务的分数进行总体平均。 +1
- **绝对提升 (Δ_{abs})**：即**“有技能”状态下的通过率**减去**“无技能（基线）”状态下的通过率**，直观展示技能到底带来了多少百分点的性能提升。 +1
- **归一化增益 (Normalized Gain, g)**：为了解决评估中的“天花板效应”（例如，一个模型从 90% 提升到 95%，和另一个模型从 10% 提升到 55%，两者的绝对提升不同，但意义都很大），论文引入了物理教育研究中常用的 Hake 归一化增益公式： +1

$$g = \frac{pass_{skill} - pass_{vanilla}}{1 - pass_{vanilla}}$$

1. 成本与计算效率指标 (Cost and Token Efficiency)

在实际部署智能体时，成本是一个关键因素。论文不仅看模型能不能做对，还看花了多大代价：

- **Token 消耗量 (Token Usage)**：统计了每次试验的平均输入 (Input)、输出 (Output) 和总 Token 数。分析发现，虽然引入 Skills 会增加 6-13% 的输入 Token（针对 Codex 和 Gemini），但其带来的性能提升远超这部分边际成本。 +1
- **单次试验估算成本 (Estimated Cost/Trial)**：根据各个模型的 API 定价计算实际花销。 +1
- **成本-性能帕累托前沿 (Cost-Performance Pareto Frontier)**：评估在相同成本下，使用 Skills 能否带来更高的性价比。例如，Gemini 3 Flash 使用 Skills 后虽然发起了更多轮次的探索（Token 消耗是 Pro 的 2.3 倍），但由于其单价极低，单次任务的成本反而比不带 Skills 的